

Relational Databases and SQL I

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercises

Exercise 0: Environment Setup

Before starting, ensure that you have SQLite installed on your system.

Option A: Manual Installation

If you are using a Debian-based operating system (like Ubuntu), you can install it using apt:

1. Update your package list:

```
sudo apt update
```

2. Install SQLite3:

```
sudo apt install sqlite3
```

3. Verify the installation:

```
sqlite3 --version
```

Option B: Docker (Recommended)

If you have Docker installed, you can use the pre-configured environment in the 02_support folder: 1. Navigate to 02_support. 2. Run `docker compose up -d`. 3. Access the Python container: `docker compose exec python-lab bash`.

Exercise 1: SQLite with Python

In this class, we will use Python's built-in `sqlite3` module to interact with an SQLite database.

1. Create a new Python file named `university_db.py`.
2. Import the `sqlite3` module and establish a connection:

```
import sqlite3

# Connect to (or create) the database file
conn = sqlite3.connect('university.db')
cursor = conn.cursor()

# Your code will go here

conn.close()
```

Exercise 2: Defining the University Schema (DDL)

Using `cursor.execute()`, create the following tables based on the University scenario:

1. University (id, name)
2. Department (id, name, univ_id)
3. Teacher (id, name, dept_id)
4. Course (id, name, dept_id, teacher_id)
5. Student (id, name)
6. Enrollment (stud_id, course_id)

Example:

```
cursor.execute('''
CREATE TABLE IF NOT EXISTS University (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL
)
''')
conn.commit()
```

Exercise 3: Inserting Data (DML)

Insert sample data into your tables. Try to use parameterized queries to prevent SQL injection (even though we are using local data).

1. Insert 1 University.
2. Insert 2 Departments.
3. Insert 3 Teachers.
4. Insert 4 Courses.
5. Insert 5 Students and enroll them in various courses.

Example:

```
teachers = [('Dr. Smith', 1), ('Prof. Jones', 1), ('Dr. Taylor', 2)]
cursor.executemany('INSERT INTO Teacher (name, dept_id) VALUES (?, ?)', teachers)
conn.commit()
```

Exercise 4: Querying Data (DQL)

Write Python code to execute and print the results of the following queries:

1. Select all teachers and their respective department names.
2. Find all students enrolled in a specific course (e.g., 'Relational Databases').
3. List courses offered by a specific department.
4. Update a teacher's name.
5. Delete a course and observe if referential integrity (foreign keys) is enforced.

Tip: To see results, use `cursor.fetchall()`.

Exercise 5: Transaction Control (TCL)

Demonstrate the "All or Nothing" property:

1. Start a transaction.
 2. Try to insert a student and an enrollment.
 3. Purposefully cause an error (e.g., insert into a non-existent table).
 4. Use `conn.rollback()` in the except block and verify that the student was NOT added.
-

Exercise 6: Normalization Challenge

Consider the following unnormalized table `raw_data`:

StudentName	Course	Instructor	InstructorOffice	Grade
Alice	Databases	Dr. Smith	Room 101	A
Alice	Physics	Dr. Brown	Room 202	B
Bob	Databases	Dr. Smith	Room 101	C

1. Identify the redundancies and potential update anomalies.
2. Decompose this table into 3NF (Third Normal Form) using the tables we defined in Exercise 2.
3. Write a Python script that reads data from a list of tuples (representing the table above) and populates your normalized tables.